

WEC7 Port: Mobvoi TicWatch C2

Codename: WEC7-C2 Target Architecture: ARM32 / ARMv7-A Status: Exploratory / BSP Design

Primary Technical Objective

Port **Windows Embedded Compact 7 (WEC7)** to the Mobvoi TicWatch C2 as a native ARM32 embedded OS. The primary first success criterion is execution of a real WEC7 kernel on physical hardware with debug diagnostics. A complete smartwatch UI and advanced connectivity represent later phase milestones.

1. TARGET HARDWARE PROFILE

Component	Specification Details	Reference & Notes
Device Target	Mobvoi TicWatch C2 (Codename: skipjack)	Unlocked bootloader enabled
SoC Platform	Qualcomm Snapdragon Wear 2100 (MSM8909W)	Quad-core ARM Cortex-A7 class
Architecture	ARM32 / ARMv7-A	32-bit Native Execution Target
System RAM	512 MB Physical RAM	Standard base memory profile
Display Panel	~360 x 360 pixel resolution	Round Wearable Panel Target
Reference OS Set	AsteroidOS & Void Linux	Hardware registers & reference drivers

2. PRIMARY SOFTWARE TARGET & SCOPE

Selected Platform

WEC7 ARM32

Windows Embedded Compact 7 is chosen because it is a lightweight, hard real-time embedded operating system designed specifically for constrained ARM hardware with full support for custom Board Support Package (BSP) development.

Explicit Non-Targets

Out of Scope

The project deliberately excludes at this stage: *Windows 10 ARM64, Windows 10 ARM32 Desktop, Windows PE environments, or qemu/software emulation targets.*

3. SEQUENTIAL SUCCESS CRITERIA

M1. BSP Buildability

Build a C2-specific WEC7 BSP toolchain.

M2. Kernel Loading

C2 bootloader loads the WEC7 ARM kernel.

M3. Kernel Execution

Execution reached with serial/debug output.

M4. Core Hardware

Bring up RAM, interrupts, timers, GPIO, clocks.

M5. Input & Storage

Touchscreen, buttons, and persistent storage.

M6. Power Management

Battery, charger, watchdog, suspend/resume.

4. BSP COMPONENT ARCHITECTURE

The Board Support Package (BSP) comprises core system initialization and OEM Adaptation Layer (OAL) modules required to bridge WEC7 with Qualcomm Snapdragon Wear 2100 hardware:

A. Boot & Startup ARM startup code, CPU initialization, kernel entry handoff, boot argument handling, memory parameters.	B. Memory Map 512MB RAM physical/virtual layout, reserved regions, framebuffer and driver memory reservations.	C. OAL Layer OEM Adaptation Layer for system init, platform interrupt routing, timers, and basic services.
D. Interrupts Qualcomm MSM8909W interrupt controller identification, routing setup, and WEC7 mapping.	E. System Timers Hardware timer identification, frequency config, and core OS scheduler timing services.	F. GPIO Controller GPIO pin mapping for display, input, power management, and communication buses.
G. Clocks & Resets Clock tree initialization, domain gating, and system reset management routines.	H. Early Debug Reliable early-boot debugging interface (UART / low-level log) setup prior to driver bring-up.	I. Display Driver Display controller, panel setup, framebuffer alignment, 360x360 pixel updates.
J. Touch Input I2C/SPI panel controller interface, touch interrupts, coordinate scaling & translation.	K. Storage Stack eMMC / storage block drivers, partition handling, and persistent filesystem access.	L. Power / Battery PMIC battery reporting, charger IC, hardware watchdog timer, suspend/resume states.

5. EXISTING HARDWARE REFERENCE SOURCES

Reference Source	Provided Information & Artifacts	Usage Policy
AsteroidOS C2 / skipjack	Device tree source (.dts), kernel source, platform drivers, boot parameters, board descriptions.	Hardware & register reference ONLY. Linux code is not directly reusable as WEC7 binaries.
Void Linux Environment	Live runtime diagnostics: dmesg, /proc layout, /sys hierarchy, active module configs.	Verification of hardware addresses and power-on state behavior.
Qualcomm MSM8909/W	Register definitions, clock specifications, pinout muxing, peripheral controller documentation.	Specification reference for custom OAL implementation.
WEC7 Reference BSPs	Standard BSP folder structure, OAL scaffolding, ARM startup conventions, driver interfaces.	Starting point scaffolding. Does not natively support MSM8909W.

6. DEVELOPMENT ROADMAP (PHASES M0 - M10)

- M0 - Recovery Plan:** Backup original partitions (boot/recovery/system). Document fastboot/bootloader restore procedures.
- M1 - Hardware Mapping:** Compile full register map for CPU, RAM, bootloader, interrupts, timer, GPIO, display, touch.
- M2 - Development Toolchain:** Establish legacy WEC7 Platform Builder environment separate from Debian host.
- M3 - C2 BSP Skeleton:** Create workspace structure, base OAL configuration, linker scripts, and startup assembly.
- M4 - First Kernel Boot:** Achieve deterministic WEC7 kernel execution and serial debug output on physical hardware.
- M5 to M10 - Peripherals & Shell:** Sequential bring-up of display (M5), touch (M6), eMMC (M7), PMIC (M8), wireless (M9), and custom watch shell (M10).

7. PROJECT RISK REGISTER

Identified Risk	Severity	Mitigation Strategy
Legacy WEC7 Tooling	HIGH	Isolate legacy Windows build environments inside isolated virtual instances outside main Debian host.
No MSM8909W WEC7 BSP	HIGH	Adapt standard ARMv7 reference BSP using AsteroidOS / Void Linux hardware specs as register guide.
Bootloader Incompatibility	HIGH	Perform deep boot protocol analysis before attempting flash operations. Use volatile boot execution first.
Missing Drivers	HIGH	Implement minimal driver stubs strictly aligned with current milestone goals (OAL before graphics).
Lack of Early Debug UART	MEDIUM	Investigate hardware test points, RAM log buffers, or display-based early framebuffer logging options.
Device Bricking / Software Damage	MEDIUM	Maintain full raw disk dumps of critical partitions and verify fastboot restoration paths before flashing.

8. NON-NEGOTIABLE ENGINEERING RULES

- **Rule 1:** Preserve existing working AsteroidOS and Void Linux installations.
 - **Rule 2:** Always perform raw partition backups prior to testing boot changes.
 - **Rule 3:** Unlocked bootloaders do not guarantee brick recovery—exercise caution.
 - **Rule 4:** Never flash unverified full image builds as a initial test.
 - **Rule 5:** Use Linux sources purely as hardware references, not binary code.
- **Rule 6:** Test the smallest atomic boot-stage changes possible.
 - **Rule 7:** Ensure all BSP build scripts and environment rules remain reproducible.
 - **Rule 8:** Explicitly record all hardware assumptions and magic register addresses.
 - **Rule 9:** Strictly decouple hardware discovery from driver implementation.
 - **Rule 10:** Kernel boot diagnostics take priority over visual UI work.

9. IMMEDIATE EXECUTION TASK & CURRENT STATUS

Current Status: EXPLORATORY / BSP DESIGN

Proven Factors: Unlocked bootloader present; hardware capable of running ARM Linux (AsteroidOS / Void Linux); 512MB RAM validated; core target is ARM32 WEC7.

Unproven Factors: WEC7 build toolchain readiness; bootloader capability to parse/pass control to WEC7 NK.bin image; initial kernel execution feasibility.

Immediate Next Task: Extract Hardware Map

Extract and document the complete C2 / skipjack hardware profile from the AsteroidOS kernel tree and runtime Void Linux instance.

Output Deliverable: Hardware map covering CPU, base RAM, boot parameters, interrupt routing, hardware timers, GPIO assignments, clock domains, display controller, panel specs, touch IC, storage partitions, PMIC, and debug pinouts. This hardware map forms the binding specification for the WEC7 BSP/OAL.

